

What Happens When Python Runs?

CPSC 250

The Big Picture

When you run

```
python myprogram.py
```

Python is:

- Not directly executed by the CPU
- Not translated into C++ source code
- Executed through an interpreter and virtual machine

Step 1: Read the Source File

Example:

```
x = 5  
y = 7  
print(x + y)
```

The Python interpreter first reads the text file.

Step 2: Parsing

The interpreter converts the text into an Abstract Syntax Tree.

Assignment

```
Variable("x")
```

```
Integer(5)
```

Step 3: Compile to Bytecode

The AST is converted into Python bytecode.

```
LOAD_CONST 5  
STORE_NAME x  
LOAD_CONST 7  
STORE_NAME y  
LOAD_NAME x  
LOAD_NAME y  
BINARY_OP +  
CALL print
```

Bytecode is lower-level than Python source, but it is still not machine code.

Step 4: The Python Virtual Machine

The bytecode is executed by the Python Virtual Machine.

```
while True:
    instruction = next_bytecode()
    execute(instruction)
```

The virtual machine acts like a tiny computer whose instruction set is Python bytecode.

Where Does C Come In?

CPython is written mostly in C.

For example:

```
x + y
```

may ultimately call something like:

```
PyNumber_Add(x, y)
```

The CPU executes the compiled C interpreter, not the original Python source.

Python Objects in Memory

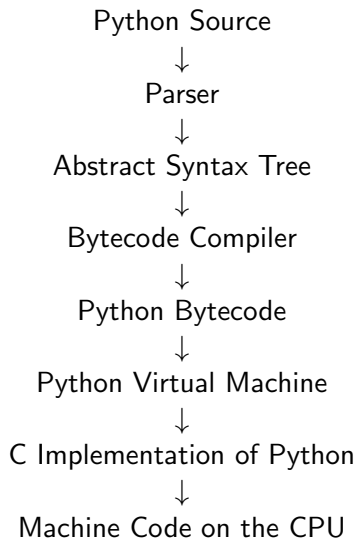
Everything in Python is an object.

```
x = 42
```

creates an integer object, and the name `x` refers to that object.

```
x ----> PyLongObject(42)
```


The Complete Pipeline



Takeaway

When you run a Python program:

- ➊ Python source code is parsed
- ➋ An AST is created
- ➌ The AST is compiled to bytecode
- ➍ The virtual machine executes the bytecode
- ➎ The interpreter's compiled C code runs on the CPU

Python is often called interpreted, but CPython also performs a compilation step.